

KAMPASS — Technical Documentation

Product: KAMPASS · Your KAM Assistant (repository: [Kratos](#)) **Scope:** pSpark 2026 · Use case 5 · Digital Key Account Management for Psiog **Release covered:** v1.3 (commit [2abf93f](#))

This document is for engineers who will run, change or review the code. For what the release contains and how to use it, see [RELEASE.md](#). Design rationale is in [decisions.md](#), security controls in [security.md](#), and the full HTTP contract in [api-contract.md](#).

1. System overview

KAMPASS replaces per-account Excel KAM workbooks with a living, scored, versioned account plan. The plan mirrors the workbook tabs S1–S12. AI agents recommend next steps, turn customer conversations into actions, brainstorm goals and answer questions. Deterministic code owns every score, risk flag and security finding.

```
Browser (React SPA, Azure Static Web Apps)
|   HTTPS, JSON, Bearer JWT
▼
Kratos.Api (ASP.NET Core 10, Azure App Service)
|   endpoints → Application services (use cases, access scope, AI agents)
|   └─ Kratos.Domain (entities, ScoringEngine; no framework dependencies)
|   └─ Kratos.Infrastructure (EF Core DAL, AI provider adapters, ClosedXML, PBKDF2)
|       └─ SQLite (local and current Azure deployment) / Azure SQL (production target)
|       └─ Gemini · Claude · Groq · Ollama (through one AI gateway with fallback)
|       └─ Key Vault (production secrets, via managed identity)
Kratos.Functions (Azure Functions timers) → same Application services
```

Design principles

- **AI explains, deterministic code decides.** Scores, risk levels, goal status, staleness and security findings are computed by rules. The AI writes recommendations, explanations and drafts.
- **Confirm before apply.** No AI output changes a plan until a person accepts it. Accepted proposals go through the same validated services the UI uses.
- **Single policy point for access.** [AccessScope](#) decides what each user can see and change. Accounts outside your scope return 404, so their existence doesn't leak.
- **Clean Architecture.** The Domain has no dependencies, the Application layer depends only on interfaces, and Infrastructure is the only code that touches the database, files or external APIs.

2. Repository layout

```
Kratos/
├── src/
│   ├── Kratos.Domain/           Entities.cs, Enums.cs, Scoring/ScoringEngine.cs
│   ├── Kratos.Application/      Use cases, access scope, AI agents, DTOs
│   │   ├── Abstractions/        Repository, provider and file interfaces (the DAL boundary)
│   │   └── Ai/                  AiCore (gateway, routing), Agents, KampassAssistant,
VoiceOfCustomerAgent,
│   │   │
│   │   └── Common/              GoalCoach, SecurityAndEvaluation
│   │       ├── Contracts/       Errors (NotFound/Forbidden/Conflict/Validation), Validator, Limits
│   │       └── Services/         DTOs (request/response records)
AuthAndAudit,
│   │   └── Services/            AccessScope, AccountService, ActionsVersionsViews, AdminService,
PlanningServices,
│   │   │
│   │   └── ReportingService, ScoringService, UtcDateTimeConverter
│   └── Kratos.Infrastructure/    Data/ (DbContext, Repositories, DataSeeder), Ai/Providers.cs,
Services/ (Excel, hashing, text)
│   ├── Kratos.Api/              Program.cs, Endpoints/Endpoints.cs, Security/,
Infrastructure/HostServices.cs, Dockerfile
│   └── Kratos.Functions/         Timer jobs (security scan, reporting snapshot, agent evaluation)
├── web/                          React 18 + TypeScript + Vite front end
│   ├── public/                  staticwebapp.config.json (routing, CSP, headers), media/login-
bg.mp4, theme-init.js
│   └── src/
│       ├── api/                  Typed client per resource, react-query hooks, mocks
(VITE_USE MOCKS)
│       ├── auth/                 AuthContext, route guards, permissions (role table), session
│       └── components/           AppShell, assistant/, tour/, charts/, KpiTile, DataTable,
UserSelect, MacSelect...
│       ├── features/             about, account, accounts, actions, admin, auth, brief, errors,
executive,
│       │
│       ├── exports, goals, import, portfolio, voc
│       ├── lib/                  env, format, theme, linkedin, macDefault
│       └── styles/               tokens.css (both themes), base, components, features, workflows,
about
│       └── test/                 Vitest + Testing Library suites
├── tests/
│   ├── Kratos.UnitTests/         Scoring engine, security primitives, workbook recognition, AI JSON
parsing, diffs
│   └── Kratos.IntegrationTests/  WebApplicationFactory: auth, scoping, roles, headers, AI 503,
Excel round trip
│       ├── smoke/                smoke_test.py (43), smoke_v11.py (47 incl. v1.2), smoke_v13.py
(10)
│       └── postman/              Kratos.postman_collection.json (30 requests) + generator
├── tools/SyntheticWorkbook/      Generates the fictional 13-tab KAM workbook in samples/
├── samples/                      synthetic-KAM-Veridian-Ports.xlsx (no customer data)
├── scripts/provision-azure.sh     Azure CLI provisioning (Key Vault, SQL, App Service, Functions,
SWA)
├── .github/workflows/            ci.yml (build + test), deploy.yml (OIDC deploy)
└── docs/                         This file, RELEASE, architecture, decisions, security, api-
contract, powerbi, blueprint
```

3. Technology stack

Layer	Technology	Notes
Front end	React 18, TypeScript (strict), Vite, react-router 7, TanStack Query 5, ECharts 6	No UI kit; design tokens in CSS variables, light and dark
API	ASP.NET Core 10 minimal APIs	Built-in OpenAPI, Swagger UI at <code>/swagger</code>
Data access	EF Core 10	SQLite locally and in the current Azure deployment; SQL Server provider for Azure SQL
Excel	ClosedXML (MIT)	Import, export, template; EPPlus rejected for licensing
AI	Anthropic C# SDK (Claude), REST for Gemini, Groq and Ollama	One <code>IAiProvider</code> interface; gateway with routing and fallback
Auth	JWT (HS256) issued by the API; PBKDF2-SHA256 password hashes	Entra ID is the production upgrade path
Jobs	Hosted service locally; Azure Functions (isolated worker) in the cloud	Same Application services
Hosting	Azure Static Web Apps (web), App Service Windows B1 (API)	Key Vault and Azure SQL scripted in <code>provision-azure.sh</code>
Tests	xUnit, WebApplicationFactory, Vitest, Testing Library, Python smoke tests, Newman	
CI/CD	GitHub Actions; Azure OIDC deploy workflow	No Azure secrets stored in GitHub

4. Domain model

Main entities (`src/Kratos.Domain/Entities.cs`):

Entity	Purpose
<code>AppUser</code>	Email, display name, PBKDF2 hash, <code>Role</code> (Admin, Executive, AccountManager), active flag
<code>AccountGroup</code>	A MAC , a named bunch of accounts, with an owner
<code>Account</code>	S1–S5 fields inline (KYC, info base, vision, strategies), <code>Status</code> (Active/Retired), <code>RowVersion</code> , per-section <code>SectionUpdatedAt</code>
<code>TeamMember</code>	RACI membership on an account
<code>Stakeholder</code>	S7 stakeholder map, plus <code>IsChampion</code> and <code>LinkedInUrl</code>
<code>Opportunity</code>	S6 matrix (potential, effort, complexity, value, priority), plus <code>GoalId</code>
<code>BrickwallRating</code> , <code>ChecklistResponse</code> , <code>ResourceNeed</code>	S8, S9, S12
<code>ActionItem</code>	S10/S11 actions with owner, due date, status, <code>ParentId</code> for sub-actions, <code>Origin</code> (Manual, AiNextBestAction, VoiceOfCustomer)

Entity	Purpose
PlanVersion	JSON snapshot of the plan plus scores per version (trend and diff source)
AccountGoal	Goal on an account or a MAC: metric, horizon (OneYear/ThreeYear), baseline, target, due date
AccountComment	Comments, e.g. executive steers
AccountDocument	Document folder, with SowStatus for SOW sign-off
VocNote	Voice-of-customer note and the extracted proposals
Master data	Offering , StrategyOption , BrickwallCriterion , ChecklistQuestion , ScoringWeight
AI and security	AiRecommendation , AiCallLog , AiEvaluation , AuditEvent , SecurityFinding , ImportSession , AccountScoreSnapshot

Scoring (ScoringEngine)

All scores are 0–100 and every weight is admin-configurable (WeightKeys.Defaults):

- **Opportunity:** weighted mix of potential, low effort and low complexity.
- **Brickwall:** weighted average of rated criteria.
- **Checklist:** Essential/Desirable/Useful weights; Yes = 1, Partial = 0.5.
- **Perception:** importance-weighted (A/B/C) stakeholder perception.
- **Completion:** average completion of the S1–S12 sections.
- **Health:** weighted blend of the above, minus penalties for overdue actions (capped at 20) and stale sections (capped at 12).
- **Risk:** High if health is below 45 or there are 3 or more overdue actions. Medium if health is below 65 or any risk reason applies. Otherwise Low.

Goal status (GoalService.Evaluate)

Progress is (current – baseline) / (target – baseline). The goal is Achieved at 100%. Otherwise progress is compared with the share of the goal's time that has elapsed: OnTrack if within 10 points, AtRisk if within 30, otherwise OffTrack.

- **Tracked from live plan data:** RunRate, Pipeline, Health, NewLogos and CrossSell.
- **Updated by hand:** Custom.
- **Opportunity share:** the value as a percentage of the goal's remaining gap. Coverage is the linked pipeline as a percentage of the remaining gap.

5. Access control

AccessScope (src/Kratos.Application/Services/AccessScope.cs) is the single decision point.

Role	Sees	Can change
AccountManager	Accounts where captain or team member	Those plans, actions, documents, goals; create accounts and MACs
Executive	All accounts, read-only	Comments, SOW sign-off, reactivation, executive views
Admin	Account metadata only	Master data, users, MACs, weights, security

Rules enforced in the services, not only in the UI:

- **Retired accounts:** read-only and hidden from views.
- **SOW sign-off:** never by the uploader.
- **Assistant proposals:** re-checked against edit rights and current data before they are applied.
- **Admin:** has no access to plan content, including through chat.

6. AI layer

Gateway (AiCore.cs , AiGateway)

- Routes each task to an ordered provider/model chain from AiOptions.Routes .
- On any provider error, or a reply that isn't valid JSON when JSON is expected, it moves to the next model.
- If every failure was temporary (429, 5xx or network), it waits 2 s and tries the whole chain once more.
- It enforces a daily token budget (Ai:DailyTokenBudget) and logs every call to AiCallLog .
- Providers without native tool use get the tool results inlined into the prompt. Providers without native structured output get the JSON schema in their instructions (SchemaPrompt).

Default routes: Claude comes first where configured; Gemini runs as the chain gemini-3.8-flash → gemini-3-flash-preview → gemini-3.1-flash-lite .

Agent	File	What it does
Next best action	Agents.cs	Tool-using recommendation (plan gaps, scores, stakeholders, actions, people) → action, helpers, channel, rationale, source sections
Plan copilot	Agents.cs	Drafts text for a section; inserted only by the user
Risk sentinel	Agents.cs	Explains rule-flagged at-risk accounts
Executive brief	Agents.cs	Portfolio narrative
Workbook interpreter	Agents.cs	Maps unrecognised workbook sheets to plan fields with confidence
KAMPASS assistant	KampassAssistant.cs	Chat across the product: answers, cross-account insights, proposals (/assistant/apply); read-only for Executives
Voice of customer	VoiceOfCustomerAgent.cs	Transcript, notes or email → summary, sentiment, signals, quoted action proposals
Goal coach	GoalCoach.cs	1-year and 3-year goal brainstorming with prompts and suggestions; answers only

Agent	File	What it does
Agent evaluator	SecurityAndEvaluation.cs	Deterministic grounding checks plus an LLM judge on NBA output
Security sentinel	SecurityAndEvaluation.cs	Rules over the audit log (brute force, credential stuffing, probing, bulk export, AI abuse, config hygiene) plus an AI explanation

Prompt-injection defence: user and file content is wrapped in tags and marked as untrusted data (`AiJson.UntrustedDataRule`). Tools are read-only and scoped to one account. All model text is length-capped before it is stored or displayed.

7. HTTP API

- **Base:** `/api/v1` , 91 routes.
- **Documentation:** OpenAPI at `/openapi/v1.json` , Swagger UI at `/swagger` .
- **Errors:** RFC 7807 problem details: 401, 403, 404 (also for out of scope), 409 (stale `rowVersion`), 422 (field errors), 429, 503 (AI unavailable).
- **Contract:** the full shapes are in `api-contract.md`, sections v1, v1.1, v1.2 and v1.3.

Area	Main routes
Auth	<code>POST /auth/login</code> , <code>GET /auth/me</code>
Accounts and plan	<code>GET/POST /accounts</code> , <code>GET /accounts/{id}</code> , <code>PUT .../profile</code>
Versions	<code>POST/GET /accounts/{id}/versions</code> , <code>.../diff</code> , <code>.../scores/trend</code>
Actions and VoC	<code>GET /actions</code> , <code>PATCH /actions/{id}</code> , <code>POST /accounts/{id}/voc</code> , <code>GET /voc</code> , <code>POST /voc/{id}/accept</code>
Views	<code>/views/mac</code> , <code>/views/nbd</code> , <code>/views/ebd</code> , <code>/views/executive</code> , <code>/champions</code>
Goals	<code>GET/POST /goals</code> , <code>PUT/DELETE /goals/{id}</code> , <code>PATCH .../progress</code> , <code>GET /goals/template</code> , <code>POST /goals/brainstorm</code> , <code>GET /accounts/{id}/goal-options</code>
Documents and SOW	<code>.../documents</code> CRUD, <code>GET /documents/{id}/content</code> , <code>POST /documents/{id}/sow/approve</code>
AI	<code>.../ai/next-best-action</code> , recommendations accept/dismiss, <code>.../ai/copilot</code> , <code>/ai/risk-scan</code> , <code>/ai/executive-brief</code> , <code>/assistant/chat</code> , <code>/assistant/apply</code> , <code>/ai/usage</code> , <code>/ai/evaluations</code>
Import and export	<code>POST /imports</code> , <code>.../commit</code> , <code>/imports/template</code> , <code>/imports/mapping-guide</code> , <code>.../export</code> , <code>/exports/bd-pack</code> , <code>/exports/executive-pack</code>
Admin and security	<code>/master</code> , <code>/admin/*</code> , <code>/macs</code> , <code>/directory/users</code> , <code>/security/findings</code>

8. Configuration

Key	Purpose	Local default
Database:Provider	Sqlite or SqlServer	Sqlite
ConnectionStrings:Kratos	DB connection	Data Source=kratos.db
Jwt:SigningKey	≥ 32 bytes; startup refuses the dev key in Production	dev key
Seed:Enabled , Seed:DemoPassword	Seed fictional demo data	enabled; password in appsettings.Development.json
Ai:Keys:Anthropic , Ai:Keys:Gemini , Ai:Keys:Groq , Ai:Keys:OllamaEnabled	Provider keys (or ANTHROPIC_API_KEY , GEMINI_API_KEY , GROQ_API_KEY env vars)	from .env (git-ignored)
Ai:DailyTokenBudget	Daily AI token ceiling	1,500,000
Cors:AllowedOrigins	Allowed web origins	http://localhost:5173
KeyVault:Uri	Load secrets from Key Vault via managed identity	empty
Scheduler:Enabled	Hosted-service jobs (use Functions in Azure instead)	true
Integrations:TeamsWebhookUrl	Adaptive Card alerts for high security findings	empty

The web app reads `VITE_API_BASE` (default `/api/v1`) and `VITE_USE MOCKS=true` for a backend-free demo.

9. Running locally

```
cp .env.example .env # add GEMINI_API_KEY and/or ANTHROPIC_API_KEY
dotnet run --project src/Kratos.Api --launch-profile http # http://localhost:5080, /swagger
cd web && npm install && npm run dev # http://localhost:5173 (proxies /api)
```

Schema changes use `EnsureCreated`, and there are no migrations yet (risk R5 in security.md). After a model change, delete `src/Kratos.Api/kratos.db*` and restart; demo data reseeds automatically.

10. Testing

Suite	Command	Current result
Backend unit	dotnet test tests/Kratos.UnitTests	49 passed
Backend integration	dotnet test tests/Kratos.IntegrationTests	8 passed

Suite	Command	Current result
Web	<code>cd web && npm test (+ npx tsc --noEmit, npm run lint, npm run build)</code>	84 passed; type, lint and build clean
End-to-end, core	<code>python tests/smoke/smoke_test.py [url] [--ai]</code>	43 checks
End-to-end, v1.1 + v1.2	<code>python tests/smoke/smoke_v11.py [url] --ai</code>	47/47 against Azure
End-to-end, v1.3	<code>python tests/smoke/smoke_v13.py [url] --ai</code>	10/10
API collection	<code>npx newman run tests/postman/Kratos.postman_collection.json --env-var baseUrl=... --env-var password=...</code>	30 requests, 35 assertions

The smoke tests write test records. Reset the database before a demo (section 9).

11. Deployment (current)

Component	Azure resource	How it's deployed
Web	Static Web App <code>swa-pspark-kratos</code>	<code>npm run build</code> with <code>VITE_API_BASE</code> , then <code>swa deploy ./dist --env production</code>
API	App Service <code>app-pspark-kratos-6jti</code> (Windows, .NET 10, B1)	<code>dotnet publish</code> , zip without <code>.pdb</code> or <code>appsettings.Development.json</code> , then stop → <code>az webapp deploy --type zip --clean true</code> → start
Data	SQLite at <code>D:\home\site\kampass-v13.db</code>	New file per schema change
Secrets	App Service settings (encrypted at rest)	Moves to Key Vault once the app's managed identity is granted <i>Key Vault Secrets User</i>

Stop the app before deploying. A hot deploy on Windows App Service left locked DLLs and caused HTTP 500.30 once.

Production path: run `scripts/provision-azure.sh` (Azure SQL with Entra-only auth, Key Vault via RBAC, Functions, OIDC). Then set `Database:Provider=SqlServer`, `KeyVault:Uri`, and GitHub variables for `deploy.yml`.

12. Known limitations and next steps

1. Add EF Core migrations; move from SQLite to Azure SQL.
2. Move secrets to Key Vault (needs a role grant by the resource-group owner).
3. Replace demo JWT sign-in with Entra ID (Static Web Apps auth or MSAL). Rotate the Gemini key that was shared during the event.
4. The container build is blocked on the corporate network (TLS inspection); build images in GitHub Actions instead.
5. A `?tab=champions` deep link opens the Accounts tab; the Champions tab works on click.

6. Account-manager MAC account counts show only accounts in the user's scope.